



This Keyword in Java:

1. This keyword is non static keyword in java.
2. **this keyword** in java is a reference variable which is used to refer current object of class.

Uses of Java This Keyword:

- this keyword can be used to refer current class instance variable.
- this keyword can be used to invoke current class method.
- this keyword can be used to invoke current class constructor.
- The main purpose of using **this keyword** is to solve the confusion when we have same variable name for instance and local variables.



CJC

Complete Java Classes

by Kunal Sir

1

this can be used to refer current class instance variable.

2

this can be used to invoke current class method (implicitly)

3

this() can be used to invoke current class constructor.

4

this can be passed as an argument in the method call.

5

this can be passed as argument in the constructor call.

6

this can be used to return the current class instance from the method.



This Keyword in Case of Constructor Calling:

- It is used for calling current class constructor.
- Every constructor has explicit first line is this keyword in case of constructor call.
- If we want to make call the current class parameterized constructor then we want to write the explicit this keyword for calling that constructor.
- When want to call the constructor from this keyword, we must want follow the syntax like below-

this(); // For non-parameterized constructor

this(parameter); // For parameterized constructor

Understanding Program:

```
public class A
{
    public A ()
    {
        this("CJC");
        System.out.println("Constructor - A");
    }
}
```



CJC

Complete Java Classes

by Kunal Sir

```
public A (String s)
{
    System.out.println("Parameterized Constructor – A = "+s);
}
}
```

```
public class Test
{
    public static void main (String [ ] args)
    {
        A a = new A ();
    }
}
```

This Keyword in Case of Method Calling:

- It is used for calling current class methods.
- We can call multiple methods by this keyword.
- If we want to call the methods from this keyword, we must want follow the syntax like below-

this.methodName(); //For non-parameterized method

this.methodName(parameter); //For parameterized method



CJC

Complete Java Classes

by Kunal Sir

Understanding Program:

```
public class A
{
    public A()
    {
        System.out.println("Non-Parameterized Constructor- A");
        this.m1("CJC");
    }
}
```

```
public void m1(String s)
{
    System.out.println("Parameterized M1- A = "+s);
}
}
```

```
public class Test
{
    public static void main (String [] args)
    {
        A a = new A ();
    }
}
```



This Keyword in Case of Variable Calling:

- It is used for calling current class variables.
- We can call multiple variables by this keyword.
- If we want to call the variable from this keyword, we must want follow the syntax like below-

this.variableName;

Understanding Program:

```
public class A
{
    int x = 10;
    int y = 20;
    int z = 30;

    public A()
    {
        int x = 50;
        System.out.println("Constructor- A");
        System.out.println(x);
        System.out.println(this.x);
        System.out.println(this.y);
        System.out.println(this.z);
    }
}
```



CJC

Complete Java Classes

by Kunal Sir

```
}  
  
}  
  
public class Test  
{  
    public static void main (String [] args)  
    {  
        A a = new A ();  
    }  
}
```

Note –

For the constructor call we can write only one this keyword inside the constructor.

Suppose our class have multiple constructors and we want to call the all the constructor so here we can write only **n-1** this keyword one constructor we can left with the super keyword compulsorily.

Where – **n** → Number of constructors.

For the methods and variable call, we can write multiple this keyword inside the constructor or method.



Difference Between Super & This Keyword:

There are following differences which is given below-

Super Keyword	This Keyword
Super represents the current instance of a parent/base class.	This represents the current instance of a class.
Used to call the default, non-parameterized and parameterized constructor of the parent/ base class.	Used to call the default, non-parameterized and parameterized constructor of the same class/current class.
Use to access methods of the parent class/ base class.	Use to access methods of the parent class / base class.
Used for pointing the super class instance.	Used for pointing the current class instance.